

# Linear Regression

CMPSC 105 – Data Exploration



ALLEGHENY COLLEGE

# Goals

- Lines
- Connection to Correlation
- Examples
- Evaluating Lines

Lines

## Formula for a Line

$$y = mx + b$$

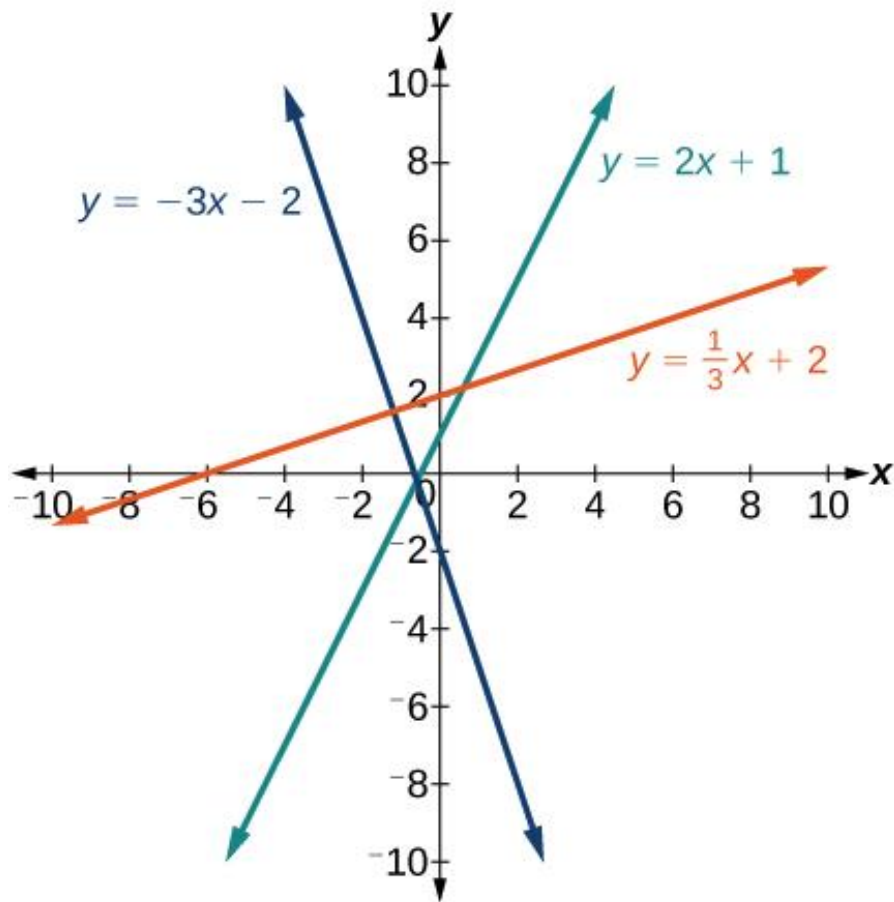
- $x$  and  $y$  are variables in a sample
- $m$  is the slope relating the two
- $b$  is the  $y$ -intercept

## Another Formula for a Line

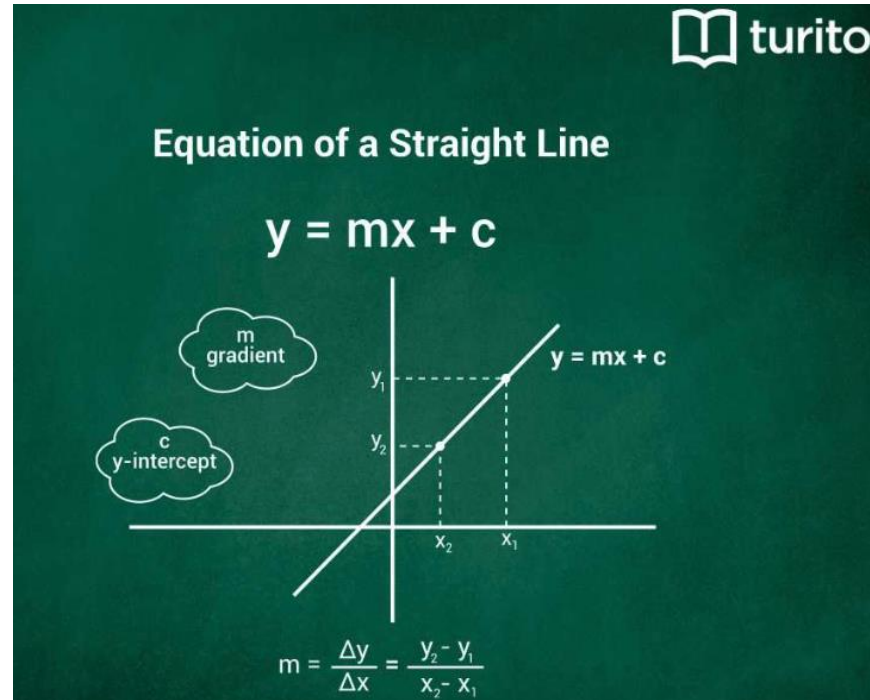
$$y = \beta_0 + \beta_1 x$$

- $x$  and  $y$  are variables in a sample with  $N$  observations
- $x$  is the **independent variable**
- $y$  is the **dependent variable**, it depends on  $x$
- $\beta_1$  is the slope
- $\beta_0$  is the  $y$ -intercept

# Slope Indicates Steepness



# Slope Describes The Relationship Between x and y



- if x and y were identical, the relationship would be 1:1
- if y never changes for any x, the relationship is 0
- sounds like a correlation!

# Connection to Correlation



# Correlation

Pearson Correction,  $r$  is defined:

$$r = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2 \times \sum_{i=1}^N (y_i - \bar{y})^2}}$$

# Slope

Slope,  $\beta_1$  is defined:

$$\beta_1 = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}$$

Together

$$r = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2 \times \sum_{i=1}^N (y_i - \bar{y})^2}}$$

---

$$\beta_1 = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}$$

## Calculating the intercept

If the slope of a line is found, then the intercept can be found

- $\beta_1 = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}$
- $\bar{y} = \beta_0 + \beta_1 \bar{x}$
- $\beta_0 = \bar{y} - \beta_1 \bar{x}$

Example

# Song Popularity Dataset

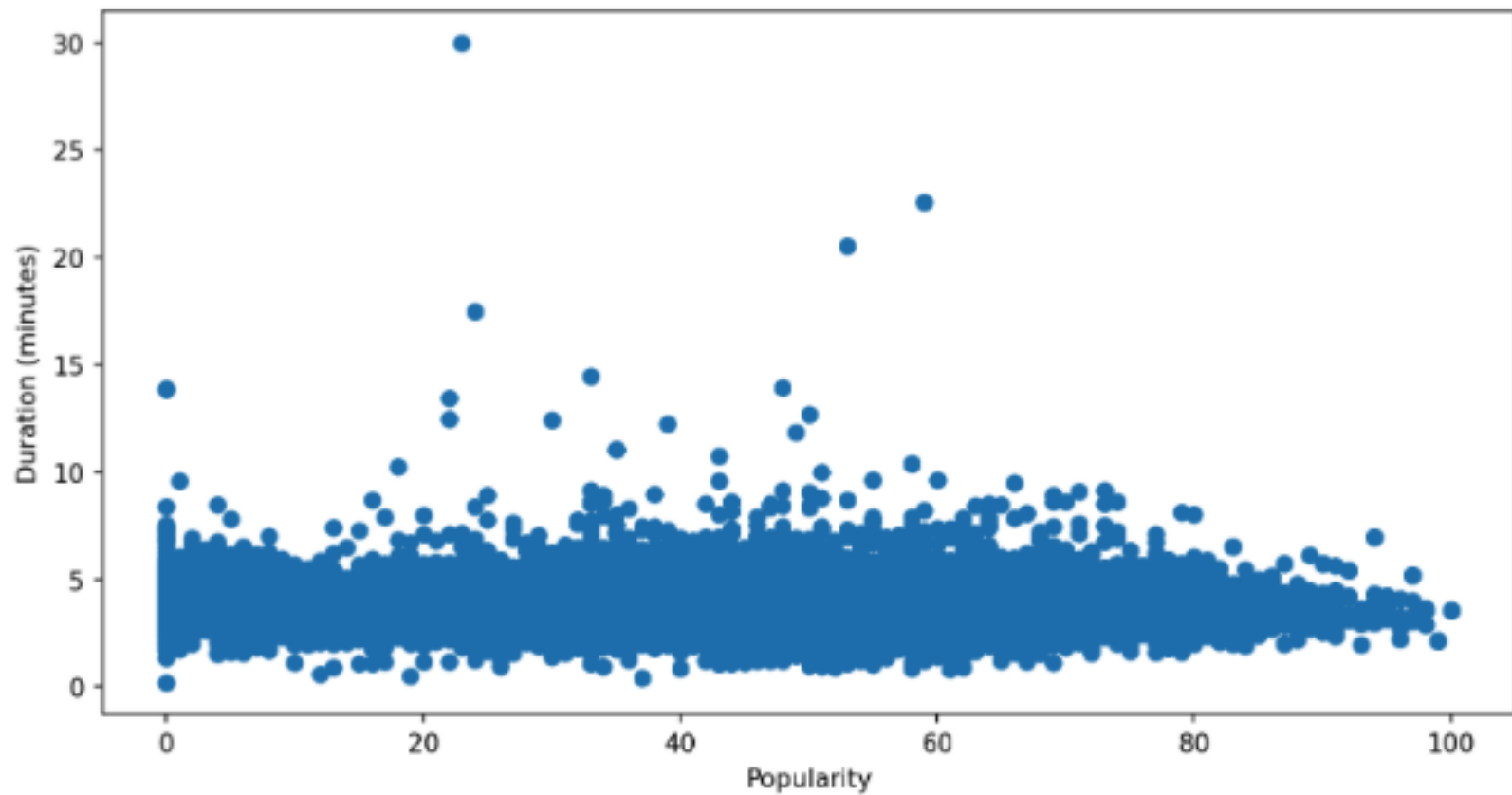
```
1 import pandas as pd
2
3 df = pd.read_csv('https://raw.githubusercontent.com,
4 df
```

|       | song_name                        | song_popularity | song_duration_ms | acousticness | danceabi |
|-------|----------------------------------|-----------------|------------------|--------------|----------|
| 0     | Boulevard<br>of Broken<br>Dreams | 73              | 262333           | 0.005520     | 0.496    |
| 1     | In The End                       | 66              | 216933           | 0.010300     | 0.542    |
| 2     | Seven<br>Nation<br>Army          | 76              | 231733           | 0.008170     | 0.737    |
| 3     | By The Way                       | 74              | 216933           | 0.026400     | 0.451    |
| 4     | How You<br>Remind Me             | 56              | 223826           | 0.000954     | 0.447    |
| ...   | ...                              | ...             | ...              | ...          | ...      |
| 18830 | Let It<br>Breathe                | 60              | 159645           | 0.893000     | 0.500    |
| 18831 | Answers                          | 60              | 205666           | 0.765000     | 0.495    |
| 18832 | Sudden<br>Love                   | 23              | 182211           | 0.847000     | 0.719    |

# Visualize

```
1 import matplotlib.pyplot as plt
2
3 pop = df.song_popularity
4 dur = df.song_duration_ms/(60 * 1000) # duration in minutes
5
6 plt.figure()
7 plt.scatter(pop, dur)
8 plt.xlabel('Popularity')
9 plt.ylabel('Duration (minutes)')
```

## Visualize





# Correlation

For all songs, what is the correlation between song popularity and song duration?

```
1 pop = df.song_popularity
2 dur = df.song_duration_ms/(60 * 1000) # duration in minutes
3
4 numerator = sum((pop - pop.mean()) * (dur - dur.mean()))
5 denominator = (sum( (pop - pop.mean())**2 ) * sum( (dur - dur.mean())**2 )) ** 0.5
6
7 r = numerator / denominator
8 print(r)
```

-0.01889940687102933

# Slope

What is the slope that fits an optimal line?

```
1 pop = df.song_popularity
2 dur = df.song_duration_ms/(60 * 1000) # duration in minutes
3
4 numerator = sum((pop - pop.mean()) * (dur - dur.mean()))
5 denominator = (sum( (pop - pop.mean())**2 ))
6
7 slope = numerator / denominator
8 print(slope)
```

-0.0008611467607479652

# Intercept

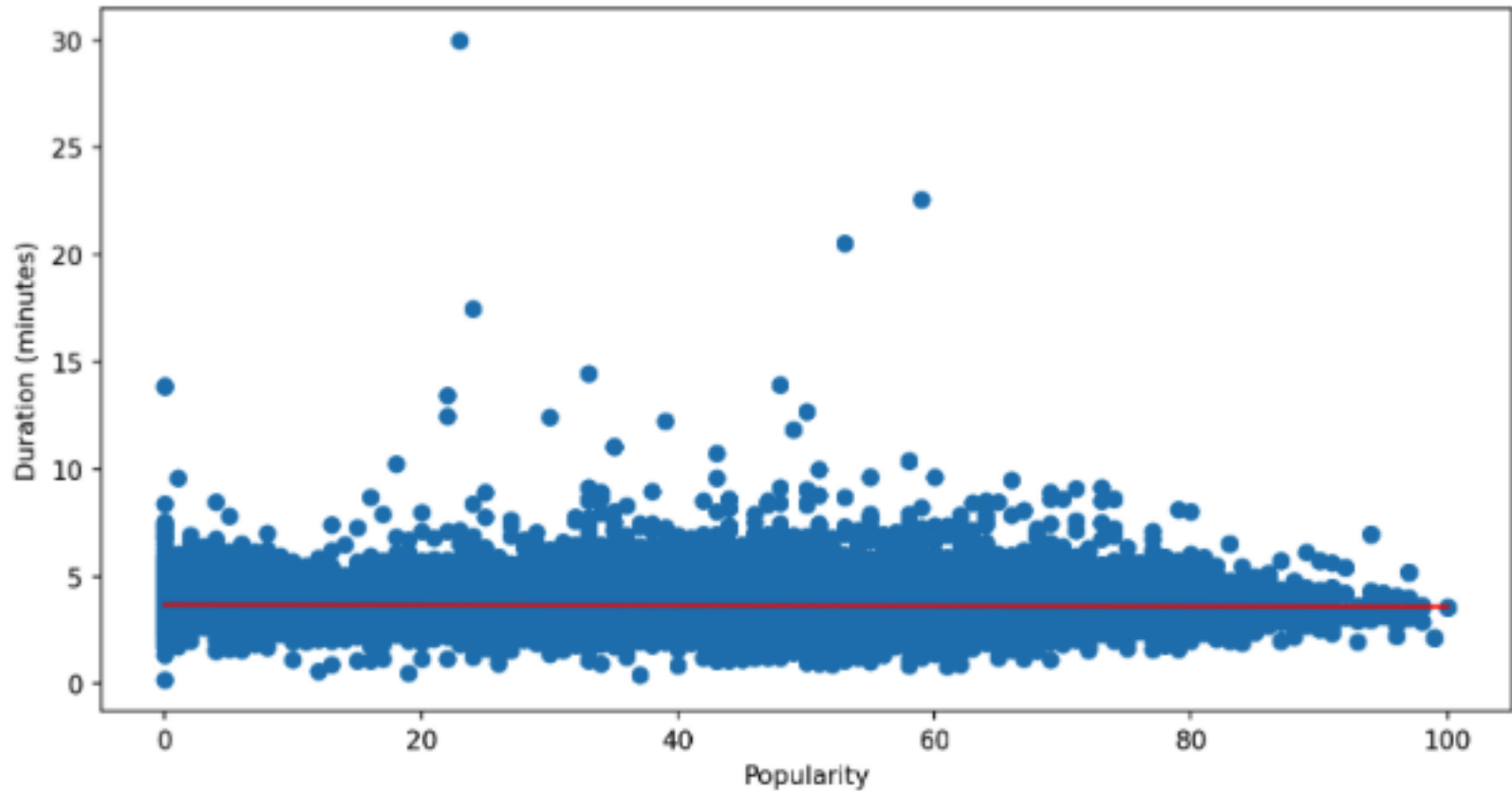
```
1 # y = m * x + b
2 # dur = slope * pop + intercept
3 # intercept = dur - slope * pop
4
5 intercept = dur.mean() - (slope * pop.mean())
6 print(intercept)
```

3.682493576012533

## Add Line to Figure

```
1 plt.figure()
2 plt.scatter(pop, dur)
3 plt.xlabel('Popularity')
4 plt.ylabel('Duration (minutes)')
5
6 plt.plot([0, 100], [slope * 0 + intercept, slope * 100 + intercept], 'r')
7 plt.show()
```

## Add Line to Figure



## Add Line to Figure

```
1 plt.plot([0, 100], [slope * 0 + intercept, slope * 100 + intercept], 'r')
```

- note how in the previous slide, I had to pick two x positions [low popularity, high popularity], then calculate
  - the y using the formula of the line!
  - for  $x = 0$ ,  $y = \text{slope} * 0 + \text{intercept}$
  - for  $x = 100$ ,  $y = \text{slope} * 100 + \text{intercept}$

# Evaluating Lines

## Evaluating Lines

- if you have an x-value and a formula for a line, the y-value can be calculated!
- in the current example, I have a lot of x-values, pop
- using pop, I can compute **PREDICTED** y-values,  $\hat{dur}$
- then I can compare  $dur$  with  $\hat{dur}$



## Comparing Predicted and Actual y-values

```
1 pred_dur = slope * pop + intercept
2 error = pred_dur - dur # compare using subtraction
3 MSE = (error ** 2).mean() # mean squared error
4 SSE = sum(error ** 2) # sum of squared error
5 print(MSE)
6 print(SSE)
```

0.9958461405203248

18756.762056700198

## Comparing Predicted and Actual y-values

- squaring errors makes everything positive, and the exaggerates large errors
- MSE is commonly used to compare models
- if this topic interests you, study **machine learning** to learn more about training & overfitting

## More Evaluation

- standard deviation for y-values would be  $\sqrt{\frac{\sum_i^N (y_i - \bar{y})^2}{N}}$
- In linear regression the numerator is the Total Sum of Squares,  $SST = \sum_i^N (y_i - \bar{y})^2$
- We saw above, the Sum of Squared Error is  $SSE = \sum_i^N (\hat{y}_i - y_i)^2$
- SST is made of SSE + SSR (Sum of Regression Errors), so  $SSR = SST - SSE$
- Or,  $SSR = \sum_i^N (\hat{y}_i - \bar{y})^2$
- Finally, the percentage of variability in y that is explained by the model is  $R^2 = \frac{SSR}{SST}$
- AND it is literally the pearson correlation squared

## More Evaluation

```
1 pred_dur = slope * pop + intercept
2 error = pred_dur - dur # compare using subtraction
3
4 SSE = sum(error ** 2) # sum of squared error
5 SST = sum((dur - dur.mean())**2) # total sum of squares
6 SSR = SST - SSE
7
8 print("this is a sanity check for SSR: ", sum((pred_dur - dur.mean())**2), SSR)
9
10 R_squared = SSR/SST
11 print("Raw R_squared: ", R_squared)
12 print("Percentage of variance explained by model: ", 100 * R_squared)
13
14 print("Pearson correlation coefficient from earlier code cell: ", r)
15 print("Pearson correlation squared: ", r ** 2)
```

this is a sanity check for SSR: 6.7020763475397525 6.70207634774124

Raw R\_squared: 0.0003571875800874598

Percentage of variance explained by model: 0.03571875800874598

Pearson correlation coefficient from earlier code cell: -0.01889940687102933

Pearson correlation squared: 0.0003571875800767106

## Using Scipy to get Probability

```
1 import scipy
2
3 slope, intercept, r, p, se = scipy.stats.linregress(pop, dur)
```

## Compare with

```
1 import scipy
2
3 r, p = scipy.stats.pearsonr(pop, dur)
```